

AI Agent 課程分支說明

本課程採用 Git 分支式教學，**16 個分支**依序展開，從 Hello World 一路到 Multi-Agent + MCP Server。每個分支只增加「一個小步驟」，可以單獨 checkout、單獨執行。

GitHub Repo: <https://github.com/kaochenlong/ai-agent-js>

技術堆疊

- **Runtime:** Node.js 22+ (純 ESM、`"type": "module"`)
- **Package manager:** npm
- **LLM 模型:** gpt-5-mini (embedding 用 text-embedding-3-small)
- **核心套件:**
 - openai v6 — OpenAI Node SDK (章節 1-3)
 - @openai/agents — Agents SDK (章節 4.1)
 - @modelcontextprotocol/sdk — MCP server (章節 4.2)
 - @inquirer/prompts — 互動輸入
 - lowdb — 對話歷史
 - ora — Loading spinner
 - zod — Tool schema 驗證
 - @qdrant/js-client-rest — 向量資料庫
 - unpdf — PDF 解析
 - csv-parse — CSV 解析
 - dotenv — 環境變數

如何使用分支

```
# 查看所有分支
git branch -a

# 切到特定分支
git checkout 1.2-openai-api

# 看當前分支跟前一個的差異
git diff 1.1-setup-env..1.2-openai-api

# 只看哪些檔案變了
git diff --name-status 1.1-setup-env..1.2-openai-api
```

階段 0：起步

0.1-hello-world

Commit: *add hello world* — 2 檔案、+4 行

目標：確認 Node.js 環境可以跑

新增檔案

```
ai-agent-course-js/
├─ hello.js
└─ README.md
```

核心程式碼

```
// hello.js
console.log("Hello World");
```

執行方式

```
node hello.js
# 輸出: Hello World
```

學習重點

- 確認 Node 22+ 已安裝
- 認識專案最小結構

階段 1：OpenAI API 基礎

1.1-setup-env

Commit: *setup environment variables with dotenv* — 6 檔案、+55 行
新增 *package.json*、*config.js*、*main.js*、*.env.example*、*.gitignore*，安裝 *dotenv*

目標：建立 npm 專案、用 dotenv 安全管理 API Key

檔案結構

```
|— .env.example
|— .gitignore
|— config.js      ← 新增
|— hello.js
|— main.js       ← 新增 (入口)
|— package.json  ← 新增
|— README.md
```

核心程式碼

```
// config.js
import "dotenv/config";

export const OPENAI_API_KEY = process.env.OPENAI_API_KEY;
```

```
// main.js
import { OPENAI_API_KEY } from "./config.js";

console.log(OPENAI_API_KEY);
```

```
# .env.example
OPENAI_API_KEY=
```

```
# .gitignore
node_modules/
.env
```

執行方式

```
npm install
cp .env.example .env    # 填入你的 OPENAI_API_KEY
node main.js
```

學習重點

- 環境變數的觀念 (API Key 不能進 git)
- dotenv/config 副作用式 import
- ESM 模組 import / export

1.2-openai-api

Commit: *add single-turn OpenAI chat completion* — 3 檔案、+456 / -3 行
安裝 *openai*、*@inquirer/prompts*; 改寫 *main.js* 完成第一次 chat completion

目標：發出第一個 Chat Completion 請求

核心程式碼

```
// main.js
import { input } from "@inquirer/prompts";
import OpenAI from "openai";
import { OPENAI_API_KEY } from "./config.js";

const client = new OpenAI({ apiKey: OPENAI_API_KEY });

const userQuestion = await input({ message: "請輸入你的問題：" });

const response = await client.chat.completions.create({
  model: "gpt-5-mini",
  messages: [
    {
      role: "developer",
      content:
        "你是一位專門講關於貓的笑話大師，請用繁體中文回答。請用幽默有趣的方式回  
應。",
    },
    { role: "user", content: userQuestion },
  ],
});

console.log(response.choices[0].message.content);
```

執行方式

```
node main.js
```

學習重點

- Chat Completions API 的訊息陣列結構
- developer / user 兩種 role
- Top-level await (ESM 才能用)
- @inquirer/prompts 取得使用者輸入

1.3-openai-api-loop

Commit: wrap chat in interactive loop — 1 檔案、+31 / -13 行
main.js 包成 while 迴圈，加上 exit 與 EOF 處理

目標：把單次對話包成持續迴圈

核心程式碼

```
// main.js
try {
  while (true) {
    const userQuestion = (
      await input({ message: "請輸入你的問題：" }).trim()
    );

    if (userQuestion === "") continue;
    if (userQuestion.toLowerCase() === "exit") {
      console.log("再會~");
      break;
    }

    const response = await client.chat.completions.create({
      model: "gpt-5-mini",
      messages: [
        { role: "developer", content: "...", },
        { role: "user", content: userQuestion }
      ],
    });
  }
}
```

```

        console.log(response.choices[0].message.content);
    }
} catch (err) {
    if (err.name === "ExitPromptError") {
        console.log("\n再會~");
    } else {
        throw err;
    }
}
}

```

學習重點

- 互動式 CLI 的 while loop
- Ctrl-C / Ctrl-D 用 ExitPromptError 處理
- 每次呼叫仍是**獨立對話**（下一個分支會加記憶）

1.4-openai-api-with-memory

Commit: *persist conversation history with lowdb* — 5 檔案、+76 / -9 行
 安裝 lowdb; 新增 db/messages.js 把對話歷史寫到
 .history/*.json; .gitignore 加 .history/

目標：用 lowdb 持久化對話歷史，讓 LLM 記得前面講過什麼

檔案結構

```

├─ db/
│   └─ messages.js      ← 新增
├─ main.js
└─ ... (其他)

```

核心程式碼

```

// db/messages.js
import { mkdirSync, existsSync } from "node:fs";
import { join } from "node:path";
import { Low } from "lowdb";
import { JSONFile } from "lowdb/node";

const HISTORY_DIR = ".history";
if (!existsSync(HISTORY_DIR)) mkdirSync(HISTORY_DIR, { recursive: true
});

```

```

const filename = `${new Date().toISOString().replace(/[:.]/g, "-")}.json`;
const adapter = new JSONFile(join(HISTORY_DIR, filename));
const db = new Low(adapter, { messages: [] });
await db.read();

export async function initMessage(systemPrompt) {
  if (db.data.messages.length === 0) {
    db.data.messages.push({ role: "developer", content: systemPrompt });
    await db.write();
  }
}

export async function addMessage(content, role = "user") {
  db.data.messages.push({ role, content });
  await db.write();
}

export function getMessages() {
  return db.data.messages;
}

```

```

// main.js (重點：呼叫時帶整個歷史)
await initMessage("你是一位專門講關於貓的笑話大師，請用繁體中文回答...");

while (true) {
  const userQuestion = ...;
  await addMessage(userQuestion);

  const response = await client.chat.completions.create({
    model: "gpt-5-mini",
    messages: getMessages(),           // ← 帶完整歷史
  });

  const content = response.choices[0].message.content;
  console.log(content);
  await addMessage(content, "assistant");
}

```

學習重點

- lowdb 簡易 JSON 資料庫

- 對話記憶 = 把每次 user / assistant 訊息全部塞回 messages 陣列
- Token 會隨對話長度線性成長（之後課程會處理）
- 三種 role: developer、user、assistant

階段 2：Tool Calling（工具呼叫）

2.1-tool-calling-1

Commit: *introduce tool calling concept with dummy weather tool* — 2 檔案、+26 行
新增 `lib/openai.js` 封裝 client、`function_call.js` 示範概念

目標：認識 tool calling 的概念，先觀察 LLM 怎麼回傳 `tool_calls`

檔案結構

```
├── function_call.js    ← 新增
├── lib/
│   └── openai.js      ← 新增
└── ...
```

核心程式碼

```
// lib/openai.js
import OpenAI from "openai";
import { OPENAI_API_KEY } from "../config.js";

export const client = new OpenAI({ apiKey: OPENAI_API_KEY });
export const DEFAULT_MODEL = "gpt-5-mini";
```

```
// function_call.js
import { client, DEFAULT_MODEL } from "../lib/openai.js";

const tools = [
  {
    type: "function",
    function: {
      name: "get_weather",
      description: "查詢即時天氣資訊",
    },
  },
];
```

```

    },
  ];

  const response = await client.chat.completions.create({
    model: DEFAULT_MODEL,
    messages: [{ role: "user", content: "請問台北現在天氣如何?" }],
    tools,
    tool_choice: "auto",
  });

  console.log(JSON.stringify(response.choices[0].message, null, 2));

```

執行結果（會看到）

```

{
  "role": "assistant",
  "content": null,
  "tool_calls": [
    {
      "id": "call_xxx",
      "type": "function",
      "function": { "name": "get_weather", "arguments": "{}" }
    }
  ]
}

```

學習重點

- Tool 不一定要實作——LLM 看到 schema 後會「決定」要不要呼叫
- `tool_choice: "auto"` 讓模型自行判斷
- 回傳的 `tool_calls` 才是 LLM 的「呼叫意圖」，需要我們真的去執行

2.2-tool-calling-2

Commit: *implement real weather tool with OpenWeather API* — 7 檔案、+322 / -15 行
 新增 `tools/weather.js` (真實 OpenWeather API)、`utils/spinner.js`; 安裝 `ora`; `function_call.js` 改為完整 tool calling 迴圈

目標：完成第一個「真實會跑」的 tool (OpenWeather)，實作完整的兩段式對話

檔案結構

```
├─ tools/
│   └─ weather.js          ← 新增
├─ utils/
│   └─ spinner.js         ← 新增
├─ function_call.js
└─ ...
```

核心程式碼

```
// tools/weather.js
import { OPENWEATHER_API_KEY } from "../config.js";

export const getWeatherTool = {
  type: "function",
  function: {
    name: "get_weather",
    description: "取得指定城市的即時天氣資訊，包括溫度、濕度、天氣狀況等。",
    parameters: {
      type: "object",
      properties: {
        city: {
          type: "string",
          description: "城市名稱（英文），如 Taipei 或 Tokyo",
        },
      },
      required: ["city"],
    },
  },
};

export async function getWeather({ city }) {
  const url = new
URL("https://api.openweathermap.org/data/2.5/weather");
  url.searchParams.set("q", city);
  url.searchParams.set("appid", OPENWEATHER_API_KEY);
  url.searchParams.set("units", "metric");
  url.searchParams.set("lang", "zh_tw");

  const res = await fetch(url);
  if (!res.ok) return { error: `OpenWeather API error: ${res.status}` };

  const data = await res.json();
  return {
    city: data.name,
```

```
    temperature: data.main.temp,
    humidity: data.main.humidity,
    description: data.weather[0].description,
  };
}
```

```
// utils/spinner.js
import ora from "ora";

export function spinner(text = "處理中...") {
  return ora(text);
}
```

```
// function_call.js (完整 tool calling 迴圈)
import { spinner } from "../utils/spinner.js";

const AVAILABLE_TOOLS = { get_weather: getWeather };
const tools = [getWeatherTool];
const messages = [{ role: "user", content: "請問台北現在天氣如何?" }];

// 第一段：問 LLM 要不要呼叫 tool
const askingSpinner = spinner("思考中...").start();
let response = await client.chat.completions.create({
  model: DEFAULT_MODEL,
  messages,
  tools,
  tool_choice: "auto",
});
askingSpinner.stop();

const message = response.choices[0].message;
messages.push(message);

if (message.tool_calls?.length > 0) {
  for (const toolCall of message.tool_calls) {
    const args = JSON.parse(toolCall.function.arguments);
    const result = await AVAILABLE_TOOLS[toolCall.function.name](args);

    messages.push({
      role: "tool",
      tool_call_id: toolCall.id,
      content: JSON.stringify(result),
    });
  }
}
```

```

}

// 第二段：把 tool 結果塞回去，請 LLM 組織人話
const replySpinner = spinner("思考中...").start();
response = await client.chat.completions.create({
  model: DEFAULT_MODEL,
  messages,
});
replySpinner.stop();
console.log(response.choices[0].message.content);
}

```

學習重點

- Node 22 內建 fetch，不必裝 axios
- Tool 結果用 `role: "tool" + tool_call_id` 塞回 messages
- 兩段式呼叫：先讓 LLM「決定」，再讓 LLM「組織答案」
- `AVAILABLE_TOOLS` 字典做動態派送
- ora spinner 包在每次 OpenAI 呼叫前後，CLI 才不會「卡住像當機」

2.3-tool-calling-3

Commit: *support multiple tools and multi-step tool loop* — 2 檔案、+64 / -26 行
 新增 `tools/youbike.js` (schema + placeholder, 2.4 才完整實作) ;
`function_call.js` 改成 while loop 支援多 tool

目標：支援多 tool、多輪 loop (LLM 可能連續呼叫多個 tool 才回答)

核心程式碼

```

// function_call.js (多 tool + 多輪 loop)
const AVAILABLE_TOOLS = {
  get_weather: getWeather,
  get_nearby_youbike: getNearbyYoubike,
};
const tools = [getWeatherTool, getNearbyYoubikeTool];

while (true) {
  const response = await client.chat.completions.create({
    model: DEFAULT_MODEL,
    messages,

```

```

    tools,
    tool_choice: "auto",
  });

  const message = response.choices[0].message;
  messages.push(message);

  // 沒有 tool_calls 就結束
  if (!message.tool_calls?.length) {
    console.log(message.content);
    break;
  }

  // 有就執行全部 tool
  for (const toolCall of message.tool_calls) {
    const args = JSON.parse(toolCall.function.arguments);
    const result = await AVAILABLE_TOOLS[toolCall.function.name](args);
    messages.push({
      role: "tool",
      tool_call_id: toolCall.id,
      content: JSON.stringify(result),
    });
  }
}

```

學習重點

- LLM 可能「先查天氣 → 再查 YouBike → 再回答」連續多輪
- 用 `while` 取代 `if`，讓 tool calling 可以無限循環直到 LLM 不再呼叫
- Tool 設計分階段：schema 先寫好讓 LLM 認識，實作可以延後

2.4-tool-calling-youbike

Commit: *implement YouBike nearby search with haversine distance* — 1 檔案、+43 / -6 行

完整實作 `tools/youbike.js`: 真實 API + Haversine 公式 + 多參數過濾

目標：完整實作 YouBike 工具，會去抓真實 API + Haversine 公式算距離

核心程式碼

```
// tools/youbike.js
```

```

const YOUBIKE_API =

  "https://tcgbusfs.blob.core.windows.net/dotapp/youbike/v2/youbike_immediate.json";

function haversine(lat1, lon1, lat2, lon2) {
  const R = 6371000; // 地球半徑 (公尺)
  const toRad = (deg) => (deg * Math.PI) / 180;
  const dLat = toRad(lat2 - lat1);
  const dLon = toRad(lon2 - lon1);
  const a =
    Math.sin(dLat / 2) ** 2 +
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.sin(dLon / 2)
  ** 2;
  return 2 * R * Math.asin(Math.sqrt(a));
}

export async function getNearbyYoubike({
  lat, lon, radius = 500, available_amount = 0, limit = 3,
}) {
  const res = await fetch(YOUBIKE_API);
  const data = await res.json();

  return data
    .filter((s) => s.act === "1")
    .map((s) => ({
      name: s.sna.replace(/^YouBike2\.0_/, ""),
      area: s.sarea,
      address: s.ar,
      available_rent: s.available_rent_bikes,
      available_return: s.available_return_bikes,
      total: s.Quantity,
      distance: Math.round(haversine(lat, lon, s.latitude,
s.longitude)),
    })))
    .filter(
      (s) => s.distance <= radius && s.available_rent >=
available_amount,
    )
    .sort((a, b) => a.distance - b.distance)
    .slice(0, limit);
}

```

範例輸出 (query 台北車站附近)

```
[
  { "name": "承德鄭州路口", "available_rent": 21, "distance": 133 },
  { "name": "捷運臺北車站(M2出口)", "available_rent": 1, "distance": 210 },
  { "name": "臺北轉運站(華陰街)", "available_rent": 6, "distance": 277 }
]
```

學習重點

- 政府開放資料 (YouBike v2 JSON API)
- Haversine 公式算地表兩點直線距離
- 串接 `.filter().map().sort().slice()` 處理陣列
- Tool 可以收多個 optional 參數, LLM 會自己決定要不要填

2.5-tool-calling-current-time

Commit: auto-generate tool schemas with zod and add current_time — 8 檔案、+83 / -62 行
新增 `utils/func-tool.js`、`tools/current_time.js`、`tools/index.js`; 重構 `tools/weather.js`、`tools/youbike.js` 改用 `zod`; 安裝 `zod`

目標: 用 **zod schema** 自動產 **tool** 定義, 消除手寫 JSON schema 的重複勞動。順便加最簡單的 tool: `get_current_time`

為什麼要這樣做? 拿同一個 weather tool 的 parameters 對照 2.2 與 2.5:

```
// 2.2: 手寫 JSON schema, nesting 一層比一層深
parameters: {
  type: "object",
  properties: {
    city: { type: "string", description: "城市名稱 (英文)" },
  },
  required: ["city"],
}
```

```
// 2.5: 改用 zod, 一行搞定
parameters: z.object({
  city: z.string().describe("城市名稱 (英文)"),
})
```

zodFunction() 會自動把 zod schema 轉成上面那種 OpenAI 期待的 JSON schema 格式（補出 type: "object"、properties、required）。schema 與函式參數同源、不會脫鉤。

檔案結構

```
├── tools/
│   ├── current_time.js    ← 新增
│   ├── index.js           ← 新增
│   ├── weather.js         ← 重構成 zod
│   └── youbike.js         ← 重構成 zod
├── utils/
│   ├── func-tool.js       ← 新增
│   └── spinner.js
└── function_call.js
```

核心程式碼

```
// utils/func-tool.js
import { zodFunction } from "openai/helpers/zod";

// 統一的 tool 物件:{ name, description, fn, parameters }
export function defineTool({ name, description, fn, parameters }) {
  return { name, description, fn, parameters };
}

// 把 tool 物件轉成 OpenAI tools 陣列要的格式
export function toOpenAITool(tool) {
  return zodFunction({
    name: tool.name,
    description: tool.description,
    parameters: tool.parameters,
  });
}
```

```
// tools/current_time.js (最簡單的範例)
import { z } from "zod";
import { defineTool } from "../utils/func-tool.js";

function getCurrentTime() {
  return new Date().toLocaleString("zh-TW", { timeZone: "Asia/Taipei"
});
}
```

```
export const currentTimeTool = defineTool({
  name: "get_current_time",
  description: "取得現在的台灣時間",
  fn: getCurrentTime,
  parameters: z.object({}),          // 無參數
});
```

```
// tools/weather.js (重構：用 zod 取代手寫 JSON schema)
export const weatherTool = defineTool({
  name: "get_weather",
  description: "取得指定城市的即時天氣資訊...",
  fn: getWeather,
  parameters: z.object({
    city: z.string().describe("城市名稱 (英文) ，如 Taipei 或 Tokyo"),
  }),
});
```

```
// tools/index.js
export { weatherTool } from "./weather.js";
export { youbikeTool } from "./youbike.js";
export { currentTimeTool } from "./current_time.js";
```

```
// function_call.js (動態組 tools)
import * as allTools from "./tools/index.js";

const toolList = Object.values(allTools);
const tools = toolList.map(toOpenAITool);
const AVAILABLE_TOOLS = Object.fromEntries(
  toolList.map((t) => [t.name, t.fn]),
);
```

學習重點

- Python 用 `inspect + docstring` 自動產 schema, JS 用 **zod**——殊途同歸
 - `zodFunction()` 是 OpenAI SDK 內建 helper (`openai/helpers/zod`) , 會把 zod 轉成 OpenAI 期待的 JSON schema
 - Tool 統一物件 { `name`, `description`, `fn`, `parameters` }, 新增 tool 只要寫一個檔
 - 設計教學重點: schema 與 function 同源, 不會脫鉤
-

階段 3：RAG（檢索增強生成）

從這章開始需要 **Qdrant** 向量資料庫。本課程用 **Qdrant Cloud** 永久免費 tier，免安裝 Docker、免信用卡。

建立 cluster 流程

1. 註冊 Qdrant Cloud → Dashboard → **Clusters** → + **Create** → 選 **Free** tier、挑一個 region
2. Cluster Detail 頁 → **API Keys** → **Create** (API key 只顯示一次，記得要複製下來)
3. 把兩個值填進 `.env`：

```
QDRANT_URL=https://<cluster-id>.cloud.qdrant.io:6333
QDRANT_API_KEY=eyJhb...
```

免費額度：1 GB RAM / 4 GB Disk，可存約 25 萬筆 1536 維向量。Netflix CSV ~8000 筆 + PDF chunk 1-2 千塊都遠在範圍內。

⚠ **重要提醒**：免費 cluster 閒置 7 天會自動 **suspend**、4 週會被刪除。

3.1-rag-text-to-vector

Commit: **embed Netflix dataset into Qdrant** — 6 檔案、+164 行
新增 `lib/qdrant.js`、`scripts/embed-netflix.js`；安裝 `@qdrant/js-client-rest`、`csv-parse`；`.env.example` 加 Qdrant Cloud 連線設定；`config.js` 加 `QDRANT_URL` / `QDRANT_API_KEY`

目標：把 Netflix 影片資料用 OpenAI Embeddings 轉成向量，灌進 Qdrant

檔案結構

```
├── data/
│   └── netflix_titles.csv    ← repo 已附
├── lib/
│   └── qdrant.js            ← 新增
└── scripts/
    └── embed-netflix.js     ← 新增
```

核心程式碼

lib/qdrant.js 從 .env 讀 QDRANT_URL 與 QDRANT_API_KEY, 連 Cloud 或本機 Qdrant 都用同一份程式碼, 差別只在 .env。

```
// lib/qdrant.js
import { QdrantClient } from "@qdrant/js-client-rest";
import { QDRANT_URL, QDRANT_API_KEY } from "../config.js";

export const qdrant = new QdrantClient({
  url: QDRANT_URL,
  ...(QDRANT_API_KEY && { apiKey: QDRANT_API_KEY }),
});

export const NETFLIX_COLLECTION = "netflix";
export const EMBEDDING_DIM = 1536;
export const EMBEDDING_MODEL = "text-embedding-3-small";
```

```
// scripts/embed-netflix.js (重點: 批次 embed + upsert)
function rowToText(row) {
  return [
    row.title, row.type, row.director, row.cast,
    row.country, row.listed_in, row.description,
  ].filter(Boolean).join(" | ");
}

async function embedBatch(texts) {
  const res = await client.embeddings.create({
    model: EMBEDDING_MODEL,
    input: texts, // ← 一次傳整個 batch (OpenAI 支援陣列)
  });
  return res.data.map((d) => d.embedding);
}

// 主流程
const rows = parse(csv, { columns: true, skip_empty_lines: true });
await qdrant.createCollection(NETFLIX_COLLECTION, {
  vectors: { size: EMBEDDING_DIM, distance: "Cosine" },
});

for (let i = 0; i < rows.length; i += 100) {
  const batch = rows.slice(i, i + 100);
  const vectors = await embedBatch(batch.map(rowToText));

  const points = batch.map((row, idx) => ({
    id: i + idx,
```

```

vector: vectors[idx],
// payload: 把原始欄位一起存，搜尋命中後才有東西可回傳（見 3.2）
payload: {
  show_id: row.show_id,
  title: row.title,
  type: row.type,
  director: row.director,
  cast: row.cast,
  country: row.country,
  release_year: row.release_year, // ← 沒進向量，但 3.2 searchNetflix
要用
  rating: row.rating,
  duration: row.duration,
  listed_in: row.listed_in,
  description: row.description,
},
}));

await qdrant.upsert(NETFLIX_COLLECTION, { wait: true, points });
}

```

加 payload vs 不加 payload

payload 在 Qdrant 是 optional 的——只給 id + vector 也能 upsert，差別在搜尋時拿回來的東西。

沒加 payload，結果只有編號跟分數：

```

[
  { "id": 1234, "score": 0.82 },
  { "id": 5678, "score": 0.79 }
]

```

你知道「第 1234 號最相似」，卻不知道那是哪部片——得再拿這些 id 回原始資料查對照，才能顯示片名與簡介。等於 Qdrant 只當「純向量索引」用，另外還要維護一份 id → 原文的對照來源。

加了 payload，搜尋一次就連原始資料一起回來：

```
[
  {
    "id": 1234,
    "score": 0.82,
    "payload": { "title": "Back to the Future", "description": "..."}
  }
]
```

`r.payload.title`、`r.payload.description` 直接可用，不用二次查詢。代價是資料冗餘（同一份 `title` / `description`，CSV 存一份、Qdrant 又存一份），但對 RAG 來說這幾乎是標配——下一步就是要把搜到的「內容」塞回去給 LLM 當 context，你要的是文字本身，不是一個 id。

執行方式

```
# 先確認 .env 已填好 QDRANT_URL 與 QDRANT_API_KEY (見階段 3 開頭)
node scripts/embed-netflix.js          # 灌資料 (會花 1-3 分鐘)
```

學習重點

- Embedding 是「把任意文字壓成 1536 維向量」
- Cosine distance: 向量空間裡的相似度
- Batch embedding (一次 100 筆) 能省 token + 加速
- `qdrant.upsert` = 沒就建立、有就更新
- Qdrant JS client 同一份程式可同時連 Cloud 與本機，差別只在 `.env` 兩個變數

3.2-rag-search-text

Commit: add semantic search over embedded vectors — 3 檔案、+46 / -24 行
`lib/qdrant.js` 加 `embed()` 與 `searchNetflix()`; 改寫 `main.js` 為搜尋介面

目標：用語意搜尋找出最相似的 Netflix 影片

核心程式碼

```
// lib/qdrant.js 新增
export async function embed(text) {
  const res = await client.embeddings.create({
    model: EMBEDDING_MODEL,
    input: text,
  });
```

```

    return res.data[0].embedding;
}

export async function searchNetflix(query, limit = 5) {
    const vector = await embed(query);

    const results = await qdrant.search(NETFLIX_COLLECTION, {
        vector, limit, with_payload: true,
    });

    return results.map((r) => ({
        score: r.score,
        title: r.payload.title,
        type: r.payload.type,
        release_year: r.payload.release_year,
        description: r.payload.description,
        listed_in: r.payload.listed_in,
    }));
}

```

```

// main.js
while (true) {
    const query = (await input({ message: "請輸入要搜尋的影片內容："
})).trim();
    if (query.toLowerCase() === "exit") break;

    const results = await searchNetflix(query, 5);

    for (const [i, r] of results.entries()) {
        console.log(`\n${i + 1}. ${r.title} (${r.type},
${r.release_year})`);
        console.log(`    分數: ${r.score.toFixed(3)}`);
        console.log(`    描述: ${r.description}`);
    }
}

```

執行方式

```

node main.js
# > 請輸入要搜尋的影片內容：時光旅行的科幻片
# > 1. Back to the Future (Movie, 1985) ...

```

學習重點

- 查詢也要先 embed 才能比對
- Cosine score 越接近 1 越相似
- with_payload: true 才能拿到原始資料 (不只是 id)

3.3-rag-tool

Commit: *wrap RAG search as a tool for the agent* — 3 檔案、+70 / -28 行
新增 `tools/netflix.js` 並加進 `tools/index.js`; 改寫 `function_call.js` 為 chatbot 雙層 loop

目標：把搜尋包成 tool，讓 LLM 自己決定「要不要查 Netflix 資料庫」

核心程式碼

```
// tools/netflix.js
import { z } from "zod";
import { defineTool } from "../utils/func-tool.js";
import { searchNetflix } from "../lib/qdrant.js";

async function search({ query, limit = 5 }) {
  return await searchNetflix(query, limit);
}

export const netflixTool = defineTool({
  name: "search_netflix",
  description:
    "在 Netflix 影片資料庫中以語意搜尋相關影片，可用於找電影、影集、紀錄片等",
  fn: search,
  parameters: z.object({
    query: z.string().describe("查詢內容，可以是劇情描述、演員、類型或關鍵字"),
    limit: z.number().default(5).describe("回傳筆數上限，預設 5"),
  }),
});
```

```
// function_call.js (改成 chatbot 並整合所有 tools)
const messages = [
  { role: "developer", content: "你是一位貼心的助理...請用繁體中文回答。" },
];

while (true) {
  const userQuestion = await input({ message: "請輸入你的問題：" });
```

```

messages.push({ role: "user", content: userQuestion });

// 內層 loop: 處理 tool calling 直到 LLM 不再呼叫
while (true) {
  const response = await client.chat.completions.create({
    model: DEFAULT_MODEL,
    messages,
    tools,
    tool_choice: "auto",
  });

  const message = response.choices[0].message;
  messages.push(message);

  if (!message.tool_calls?.length) {
    console.log(message.content);
    break;
  }

  for (const toolCall of message.tool_calls) {
    const args = JSON.parse(toolCall.function.arguments);
    const result = await AVAILABLE_TOOLS[toolCall.function.name](args);
    messages.push({
      role: "tool",
      tool_call_id: toolCall.id,
      content: JSON.stringify(result),
    });
  }
}
}

```

學習重點

- RAG = 把搜尋邏輯包成 tool，由 LLM 自主決策
- 雙層 loop：外層處理使用者每次提問、內層處理 tool calling 多輪
- 所有 tool（天氣、YouBike、時間、Netflix 搜尋）共存，LLM 自己選

3.4-rag-for-pdf

Commit: **add PDF document RAG with recursive text splitting** — 6 檔案、+144 行
安裝 `unpdf`; 新增 `scripts/embed-pdf.js`、`tools/pdf.js`; `lib/qdrant.js`
加 `PYTHON_BOOK_COLLECTION` 與 `searchPythonBook()`

目標：把 PDF 教科書切 chunk、灌進 Qdrant、做 RAG 問答

檔案結構

```
├── data/
│   ├── netflix_titles.csv
│   └── python-book.pdf      ← repo 已附
├── scripts/
│   ├── embed-netflix.js
│   └── embed-pdf.js        ← 新增
├── tools/
│   ├── ...
│   └── pdf.js              ← 新增
└── lib/qdrant.js           ← 加 searchPythonBook
```

核心程式碼

```
// scripts/embed-pdf.js (重點: PDF → text → chunks → embed → upsert)
import { extractText, getDocumentProxy } from "unpdf";
import { createHash } from "node:crypto";

function splitText(text, chunkSize = 1000, overlap = 50) {
  const chunks = [];
  let start = 0;
  while (start < text.length) {
    const end = Math.min(start + chunkSize, text.length);
    chunks.push(text.slice(start, end));
    if (end === text.length) break;
    start = end - overlap;      // ← overlap 避免切斷語意
  }
  return chunks;
}

const buffer = new Uint8Array(await readFile(PDF_PATH));
const pdf = await getDocumentProxy(buffer);
const { text, totalPages } = await extractText(pdf, { mergePages: true });

// 用 md5 hash 去重 (PDF 內容可能有重複頁面)
```

```
const seen = new Set();
const chunks = splitText(text, 1000, 50).filter((c) => {
  const h = createHash("md5").update(c).digest("hex");
  if (seen.has(h)) return false;
  seen.add(h);
  return true;
});

// Batch embed + upsert (同 3.1)
```

```
// tools/pdf.js
export const pythonBookTool = defineTool({
  name: "search_python_book",
  description:
    "在 Python 教科書中搜尋相關內容，回答 Python 語法、概念、API 用法問題時可使用",
  fn: search,
  parameters: z.object({
    query: z.string().describe("要搜尋的關鍵字或描述"),
    limit: z.number().default(5).describe("回傳筆數上限，預設 5"),
  }),
});
```

執行方式

```
node scripts/embed-pdf.js          # 灌 PDF
# 然後 node function_call.js
# > 請輸入你的問題：Python 的 decorator 是什麼？
# LLM 會呼叫 search_python_book 工具
```

學習重點

- PDF → 純文字用 unpdf (純 JS、無 native dep)
- Chunking 策略: chunkSize=1000, overlap=50
- Overlap 的用途: 避免關鍵句被切斷導致語意不完整
- Hash 去重避免重複 chunk
- 不同知識庫 = 不同 collection (netflix / python_book)

階段 4：進階 (Agents SDK + MCP)

4.1-agents-sdk

Commit: *introduce OpenAI Agents SDK with multi-agent handoff* — 5 檔案、+1508 / -82 行 (含 README + package-lock 大量變動)

安裝 @openai/agents; 新增 `utils/agent-tool.js`; 完全改寫 `main.js` 為 Agent + Runner + handoff 架構

目標: 用 @openai/agents 取代手寫 tool calling loop, 並引入多 **agent + handoff**

- 安裝 @openai/agents
- 新增 `utils/agent-tool.js`: 把現有 tool 物件包成 SDK 的 `tool()`
- 完全改寫 `main.js`: 用 Agent + Runner + handoff
- 4 個 agent: 班導師 (協調者)、PHP 老師、Vue 老師、Python 老師
- 班導師擁有所有 local tools + handoff 給三位老師
- Python 老師額外擁有 `search_python_book` tool (會用 3.4 灌好的知識庫)

核心程式碼

```
// utils/agent-tool.js
import { tool } from "@openai/agents";

export function toAgentTool(t) {
  return tool({
    name: t.name,
    description: t.description,
    parameters: t.parameters,
    execute: async (args) => {
      const result = await t.fn(args);
      return typeof result === "string" ? result :
JSON.stringify(result);
    },
  });
}
```

```
// main.js
import { Agent, run } from "@openai/agents";

const MODEL = "gpt-5-mini";

const phpTeacher = new Agent({
  name: "PHP 老師",
```

```

model: MODEL,
instructions:
  "你是 PHP 老師，專門回答 PHP、Laravel 相關問題。請用繁體中文回答。",
handoffDescription: "PHP 或 Laravel 相關問題",
});

const vueTeacher = new Agent({
  name: "Vue 老師",
  ...
});

const pythonTeacher = new Agent({
  name: "Python 老師",
  model: MODEL,
  instructions:
    "你是 Python 老師。回答問題時，請優先使用 search_python_book 工具查教科書，再用自己的話解釋。",
  tools: [toAgentTool(pythonBookTool)], // ← 只有 Python 老師能查 PDF
});

const homeroom = Agent.create({
  name: "班導師",
  model: MODEL,
  instructions: `你是班導師...
- PHP / Laravel 問題請 handoff 給 PHP 老師
- Vue.js / Nuxt 問題請 handoff 給 Vue 老師
- Python 問題請 handoff 給 Python 老師
- 一般生活問題 (天氣、時間、YouBike、Netflix) 可以直接用 tools 回答
請用繁體中文回答。`,
  tools: [
    toAgentTool(currentTimeTool),
    toAgentTool(weatherTool),
    toAgentTool(youbikeTool),
    toAgentTool(netflixTool),
  ],
  handoffs: [phpTeacher, vueTeacher, pythonTeacher],
});

while (true) {
  const userInput = await input({ message: "請輸入你的問題：" });
  const result = await run(homeroom, userInput);

  console.log(`[由 ${result.lastAgent?.name ?? "班導師"} 回答]`);
  console.log(result.finalOutput);
}

```

```
}
```

範例對話

```
> 請問現在台北的天氣？
[由 班導師 回答] 台北目前 25°C，晴朗...

> Python 的 decorator 怎麼寫？
[由 Python 老師 回答] 根據教科書... (先呼叫了 search_python_book)

> Laravel 的 Eloquent 怎麼用？
[由 PHP 老師 回答] Eloquent 是 Laravel...
```

學習重點

- Agent 取代「自己組 messages、跑 loop」的所有手工
- handoffs 讓 LLM 自己決定要轉介給誰
- 不同 agent 可以有不同 tools、不同 instructions
- `result.lastAgent` 告訴你最終回答的是誰
- 整合前面 2.x + 3.x 累積的 tools，無痛升級

4.2-mcp-server

Commit: *expose tools via MCP server* — 4 檔案、+78 / -99 行
安裝 `@modelcontextprotocol/sdk`；新增 `mcp-server.js` (`McpServer + stdio`)；`main.js` 接 `MCPServerStdio` 連線

目標：用 **MCP (Model Context Protocol)** 把 tool 抽出成獨立 server，讓任何 LLM client 都能透過標準協定使用

核心程式碼

```
// mcp-server.js
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from
"@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";

const server = new McpServer({
  name: "tenlong-tools",
  version: "1.0.0",
```

```

});

server.registerTool(
  "add",
  {
    title: "兩數相加",
    description: "把兩個數字相加並回傳結果",
    inputSchema: { a: z.number(), b: z.number() },
  },
  async ({ a, b }) => ({
    content: [{ type: "text", text: String(a + b) }],
  }),
);

server.registerTool(
  "get_tenlong_book_list",
  {
    title: "天瓏熱門書籍",
    description: "取得天瓏書店熱門技術書籍清單",
    inputSchema: { limit: z.number().min(1).max(20).default(5) },
  },
  async ({ limit }) => ({
    content: [
      { type: "text", text: JSON.stringify(BOOK_LIST.slice(0, limit),
null, 2) },
    ],
  }),
);

const transport = new StdioServerTransport();
await server.connect(transport);
console.error("[tenlong-tools MCP server] running on stdio");
//
// 注意：MCP 用 stdout 傳 JSON-RPC，所以 log 一律走 stderr

```

```

// main.js (在 4.1 基礎上加 MCP server)
import { Agent, run, MCPServerStdio } from "@openai/agents";

const tenlongMcp = new MCPServerStdio({
  fullCommand: "node mcp-server.js",
  name: "tenlong",
  cacheToolsList: true,
});

```

```
await tenlongMcp.connect();

const homeroom = Agent.create({
  // ... 同 4.1
  mcpServers: [tenlongMcp],      // ← 新增
});

try {
  while (true) { /* 同 4.1 */ }
} finally {
  await tenlongMcp.close();      // ← 記得收尾
}
```

測試 MCP server 是否正常

```
# 方法 1: 用 MCP Inspector 視覺化測試
npx @modelcontextprotocol/inspector node mcp-server.js

# 方法 2: 用 JSON-RPC pipe 手動探
echo '{"jsonrpc":"2.0","id":1,"method":"initialize",...}' | node mcp-server.js

# 方法 3: 直接跑 main.js 讓班導師使用
node main.js
# > 請推薦天瓏書店最近熱門的 3 本書
# 班導師會透過 MCP server 呼叫 get_tenlong_book_list
```

學習重點

- MCP 是 Anthropic 主推的「tool 標準化協定」，正在成為跨 LLM 的標準
- MCP server 用 **stdio** 通訊 (JSON-RPC over stdin/stdout)
- Server 端用 `registerTool(name, { inputSchema }, handler)`
- Client 端用 `@openai/agents` 的 `MCPServerStdio` 一行接入
- 同一個 MCP server 可以被 Claude Desktop、Cursor、各種 IDE 重用

總結

16 個分支從「印 Hello World」走到「Multi-Agent 透過 MCP server 共享 tool」，完整涵蓋：

- OpenAI API 基礎 (messages、roles、迴圈、記憶)

- Tool Calling (手寫 schema → zod 自動產 schema → 多 tool 整合)
- RAG (embedding、向量資料庫、PDF chunking)
- Agents SDK (多 agent、handoff、MCP server 整合)

學會這套之後，你可以：

- 把自己工作領域的資料灌成知識庫（換 3.x 的 CSV / PDF）
- 加自己領域的 tool（仿 2.5 的 zod 模式）
- 建立自己領域的 multi-agent 團隊（仿 4.1 的 handoff 設計）
- 把工具發布成 MCP server 給 Claude Desktop、Cursor 等用（仿 4.2）